

이동형

DevOps Engineer | 이력서 & 경력기술서 · 경력 4년 3개월

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

소개

안녕하세요, DevOps Engineer 이동형입니다.

인프라 가용성 99.9%를 목표로 모든 변경을 코드화(iaC)하고 장애 대응을 포스트모템으로 자산화하며 운영 효율을 높여왔습니다. 단순 운영을 넘어 업무의 의미를 묻고 더 나은 방향을 찾는 습관이 가장 큰 강점입니다.

AWS·GCP 클라우드와 OpenStack 기반 온프레미스 PaaS에 걸친 인프라를 설계·구축합니다. Kubernetes로 컨테이너 운영을 표준화하고, Terraform·Ansible로 인프라를 코드화하며, GitLab CI·ArgoCD 중심의 자동 배포 파이프라인과 모니터링·로깅 스택을 운영합니다. 반복 업무는 Python·Bash로 자동화합니다.

데이터 파이프라인 자동화와 사내 GPU LLM 서비스로 데이터 엔지니어링·AI/ML 인프라 인접 영역도 직접 운영하고, git-bridge·Slack APK 봇 같은 사내 DevOps 도구도 직접 개발합니다.

3개 회사에서 AWS→GCP 마이그레이션, AWS·온프레미스 하이브리드, 온프레미스 PaaS 구축을 단독 주도하며 AWS 비용 20%·GCP 50% 절감, 배포 시간 50~67% 단축, 수동 작업 90% 이상 자동화를 달성했습니다. 최근에는 신규 게임의 클라우드 프로덕션 런칭을 위한 AWS EKS 멀티클러스터를 구축하고 helmfile push 배포를 ArgoCD pull-GitOps로 전환했으며, 게임 DB 14초 멈춤 장애 해소와 Keycloak 중앙 SSO 통합으로 운영 효율을 한 단계 더 끌어올리고 있습니다.

앞으로도 인프라 아키텍처 고도화와 조직 생산성 향상에 기여하는 엔지니어가 되고자 합니다.

강점

본질을 묻는 사고 습관

- 업무의 본질을 묻는 습관 — ‘왜 이 작업이 필요한가’를 먼저 정의한 뒤 도구를 선택

체계적인 문서화와 지식 공유

- 모든 작업 과정을 문서로 정리하고 팀과 공유하여 조직의 지식 자산 구축에 기여

기록 기반의 지속적 개선

- 장애·인프라 변경·트러블슈팅을 포스트모템으로 기록·자산화 — 재발 방지·지속 개선

풀스택 인프라 경험

- 클라우드(AWS, GCP), 온프레미스, 네트워크, CI/CD, 모니터링을 아우르는 End-to-End 인프라 경험 보유

기술 스택

클라우드	AWS, GCP, OpenStack
컨테이너 오케스트레이션	Kubernetes, Cilium CNI, NGINX Gateway Fabric / Gateway API, Karpenter
Infrastructure as Code	Terraform, Ansible, Helm, Helmfile
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, ArgoCD ApplicationSet, Jenkins, GitOps
모니터링 & 로깅	PLG Stack, ELK Stack, EFK Stack, ECK Operator, Fluent Bit, Thanos
보안 & IAM	Keycloak (Operator, OIDC IdP), Vaultwarden, GitLab SSO (OIDC), External Secrets Operator, Sealed Secrets
자동화	Go, Python, Shell Scripting
데이터베이스	MySQL, PostgreSQL, Redis, MongoDB
OS & 서버	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS, GCP GKE 프로덕션 환경 운영 및 비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감, 절감분 신규 프로젝트 재투자)

- Terraform을 활용한 IaC 기반 인프라 자동화 및 버전 관리 (GCP 리소스 100% 코드화)
- 온프레미스-클라우드 하이브리드 아키텍처 설계 및 마이그레이션 경험
- Kubernetes 스토리지 성능 최적화 (control-plane HDD → SSD 마이그레이션으로 etcd fsync 1760ms → 3.88ms 454배 개선, GitLab Runner 빌드 I/O 격리로 게임 DB COMMIT latency 14초 → ms 수준 회복)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- GitHub Actions, GitLab CI, ArgoCD를 활용한 CI/CD 파이프라인 설계 및 고도화
- 배포 시간 50~67% 단축, 주간 배포 3배 증가로 Time to Market(시장 대응 속도) 단축, 롤백 시간 95% 감소 (30분 → 1~2분)
- Jenkins 기반 Unity Android/iOS 빌드 자동화 및 Slack 연동 배포 알림 구축
- 공용 Toolchain Image + 자동 버전 사이클 구축으로 15개 consumer repo 표준화, CI 첫 번째 잡 설치 시간 5분 → 약 10초(약 95% 단축), 사람 클릭 2회로 4단계 연쇄 전파 자동화

모니터링 및 오피서빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- PLG Stack (Prometheus, Loki, Grafana) 기반 메트릭·로그 통합 모니터링 구축
- ELK → EFK Stack 전환 후 ECK Operator 기반 Elastic Stack 9.0 CRD 선언형 관리로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- SLI/SLO 기반 알림 체계로 MTTR(평균 장애 복구 시간) 단축 및 서비스 가용성 향상
- kube-prometheus-stack 기반 다수 커스텀 알림 규칙·여러 Grafana 대시보드·4종 DB exporter (MySQL, Redis, Elasticsearch, PostgreSQL) 통합
- Fluent Bit 로그 수집기 데이터 무결성·HA 강화 (SQLite offset DB + PVC, PodDisruptionBudget + preStop hook, 7개 Prometheus alert로 Pod 재시작 시 로그 누락/중복 0건 달성)

자동화 및 효율화

Python, Bash를 활용한 운영 자동화 및 개발 생산성 도구 구축

- 데이터 수집, 배포 프로세스, 문서 관리 자동화로 수동 작업 90% 이상 감소
- 내부 LLM 서비스(Ollama + Open WebUI) 구축으로 사내 보안 가이드라인 100% 준수 및 개발팀 생산성 향상
- GitLab Webhook 기반 문서 자동 동기화, Slack Bot 기반 APK 배포 알림 등 사내 도구 개발
- Shell → Python(stdlib only) 마이그레이션 (758 unittest) + 25개 컴포넌트 단계별(wave) 순차 자동 배포·업그레이드 MR 자동 생성
- DP사 운영팀 Kubernetes 자체 운영 체계 확립 — 6개월 교육 프로그램 설계·진행 (수강자 평균 10명·회당 4시간), 교육 자료 표준화로 타 고객사 교육에도 재활용

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택의 무중단 전환과 재사용 가능한 공용 차트 오픈소스 공개

- Ingress-nginx 다수 인스턴스 → NGINX Gateway Fabric 단일 컨트롤러인 + 클래스별 Gateway CR 무중단 마이그레이션 (전체 cutover, wildcard TLS 통합)
- ECK Operator 기반 로깅 스택을 로컬 차트 → OCI 차트로 무중단 전환 (CR/PVC 이름 보존, cluster_uuid 불변, 데이터 손실 없음)
- 오픈소스 Helm 차트 3종(nginx-gateway-cr, elasticsearch-eck, kibana-eck) 및 composite GitHub Actions 다수 공개, ArtifactHub / GitHub Marketplace 등록

인프라 보안 & IAM

Keycloak Operator 기반 중앙 인증(SSO) 도입과 Vaultwarden 비밀번호 관리 시스템으로 사내 인증·시크릿 거버넌스 일원화

- Keycloak Operator + KeycloakRealmImport로 중앙 인증 시스템 도입 — GitLab brokering으로 기존 계정 그대로 로그인, ArgoCD / Harbor / Vaultwarden OIDC 통합(다운타임 0, 검증 PASS), GitLab group → Keycloak mapper → ArgoCD role 단일 권한 흐름 확립
- Vaultwarden을 Kubernetes에 Helm 배포, GitLab SSO + 자동 백업(SQLite 덤프 30일 retention) + 대화형 restore 스크립트로 사용자 70명·시크릿 50건 중앙 관리
- git-bridge Retry API — webhook secret 노출 0 + SRE 친화적 수동 복구 경로 (Bearer 인증·repo-level direction override)

데이터 엔지니어링 및 사내 LLM 서비스

GCP 데이터 파이프라인부터 사내 LLM 서비스까지 직접 구축·운영

- BigQuery + Dataflow + Cloud Functions + Cloud Scheduler 조합으로 멀티 데이터 소스(MongoDB·CloudSQL·GA·Dune) → BigQuery → Google Sheets 자동 파이프라인 구축, 인프라 100% Terraform IaC
- 온프레미스 GPU 서버 위 Ollama + Open WebUI 기반 사내 LLM 서비스 Kubernetes 배포·운영, 외부 AI SaaS 구독 대체로 월 약 \$100 절감 및 사내 보안 가이드라인 100% 준수
- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0% 달성

주요 경력 상세 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. 개발 서버와 테스트 서버는 온프레미스에, QA와 Production 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중이며, 이 구조로 월 인프라 비용을 20%(\$60,000 → \$48,000) 절감하고 절감분을 신규 프로젝트에 재투자하는 선순환을 만들었습니다.

현재 서비스 중인 라이브 게임에 대해 외부 퍼블리셔와 협업하여 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도 100% (단독 설계 및 구현, 외부 퍼블리셔 운영팀 협업) 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용 분석을 수행한 결과, 개발·테스트 환경은 트래픽 변동이 적어 클라우드의 자동 스케일링이 불필요했고, QA·Production 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 개발·테스트 환경은 온프레미스로 전환하고, QA·Production 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다.

양쪽 환경은 VPN 없이 역할을 분리해 독립 운영(앱 트래픽 분리)하며, 이미지 레지스트리도 환경별로 분리(개발=Harbor, QA·Production=ECR)해 아키텍처 차이에 맞춰 각 환경에서 독립적으로 빌드했습니다. 양쪽 환경 모두 Terraform과 Ansible을 활용한 IaC 통합으로 운영 표준을 일원화(신규 컴포넌트 IaC 100%, IAM 제외)하여, 운영 복잡도 증가 없이 일관된 인프라 관리를 실현했습니다.

트러블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
- Kubernetes IPVS 모드에서 ClusterIP 서비스 간 통신 시 간헐적 타임아웃 발생. IPVS conntrack 임계치 튜닝으로 해결
- ArgoCD 로 관리하는 프로덕션 서비스들의 rolling deploy 중, ALB target deregistration delay 와 Kubernetes Pod 의 SIGTERM 처리 시점이 어긋나면서 일부 세션 connection drop 발생. 다음 3가지 조치로 rolling deploy 중에도 connection drop 0건·zero-downtime 배포 달성:
 - (1) ALB drain 타이밍 정렬 — target group deregistration_delay 를 기본 300초에서 30초로 단축하고, Pod lifecycle.preStop hook 에 sleep 35s 추가. 종료 중 readiness probe 가 먼저 fail 하여 ALB 가 target 을 unhealthy 로 인식·신규 요청을 차단하되, in-flight 요청은 계속 처리해 무중단 확보. terminationGracePeriodSeconds 는 preStop(35s)을 초과하도록 설정 — 예를 들어 앱이 SIGTERM 시 in-flight 요청을 자체 drain(~30초)하는 서비스는 preStop 35s + drain 여유까지 합산해 75초, 그 외 서비스는 60초
 - (2) RollingUpdate 전략 조정 — maxSurge 는 절대값 1로 고정(퍼센트 지정 시 노드가 다수 동시 생성되므로 절대값으로 제어)하고, maxUnavailable 은 replica 스케일에 맞춰 설정 — 예를 들어 replica 2~3이면 maxUnavailable 1로 surge-first(롤아웃 중 ~100% 용량 유지), replica 10이면 "50%"→0 반올림으로 surge-first zero-downtime, 대규모(replica 10~20)면 maxSurge 0 + maxUnavailable 20~25%로 신규 노드 생성 없이 배치 롤아웃
 - (3) 다중 replica(2 이상) 서비스에 PDB 적용 — Karpenter 노드 consolidation 시 Pod 을 하나씩만 축출하도록 보호 (단일 replica 서비스는 PDB 미적용)

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 절감분을 신규 프로젝트 인프라에 재투자하여 리소스 선순환 구조 구축
- 개발 서버 온프레미스 이전으로 월 약 \$1,000 AWS 리소스 비용 추가 절감 및 리소스 사용량 최적화
- 환경별 역할 분리로 운영 안정성 확보 및 장애 격리

기술 스택

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

프로젝트 2: 온프레미스 CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. Kaniko를 활용한 Docker 이미지 빌드로 Docker-in-Docker 의존성을 제거하고, 멀티 환경(alpha/review/dev/staging) 배포를 단일 파이프라인에서 관리하도록 구성했습니다.

트러블슈팅

- ArgoCD Application 의 `syncPolicy.automated.selfHeal=true` 가 운영자의 임시 hot-fix 를 즉시 되돌려 장애 디버깅이 어려워지는 문제 발생. prod 는 `selfHeal=false` + manual sync window 로 분리하고 dev/staging 만 `selfHeal=true` 유지, 모든 sync 이벤트 Slack 알림 연동으로 변경 추적성과 안정성 동시 확보
- GitLab 업그레이드 후 GPG 키 만료로 CI Runner에서 패키지 설치 실패. apt-key 갱신 절차를 runbook으로 표준화·문서화하여 재발 시 신속 대응
- Kaniko 빌드 시 캐시 무효화로 빌드 시간 급증. `-cache=true --cache-ttl=24h --snapshot-mode=redo` 옵션 조합으로 해결
- 데이터 배포 파이프라인 소스 페치 병목 점진적으로 튜닝 — 기존 `git checkout + git pull` 이 매 실행마다 전체 history(약 3.5GB)를 다시 fetch해 약 139초 소요되던 것을 `git fetch --depth=1 --filter=blob:none` (tip-only shallow + blob 필터)로 교체해 소스 페치 시간 대폭 단축
- npm 의존성 설치를 `npm ci --cache .npm --prefer-offline` + lockfile 해시 cache key로 tarball 다운로드 재사용(의존성 변경 시에만 무효화), 데이터 업로드는 SSH scp/rsync 직결을 S3 transit 버킷 스테이징 → SSM SendCommand 기반 `aws s3 sync --exact-timestamps` 로 전환해 SSH 키 제거 및 배포 계정 키 유출 리스크 차단

성과

- 배포 시간 67% 단축 (30분 → 10분), 수동 작업 자동화 90% 달성
- 배포 자동화로 주간 배포 횟수를 3배 이상으로 늘려(1~2회 → 5~7회) 신규 피처의 Time to Market 단축, 배포 리소스 효율화로 개발팀이 기능 개발에 집중할 수 있는 환경 조성
- 크로스레포 MasterData 파이프라인 오케스트레이션 — 데이터 레포 변경 시 하위 서비스로 자동 fan-out(일부 서비스는 전 환경, 일부는 dev/qa 한정), 수신 측 파이프라인이 upstream 아티팩트를 받아 실제 변경 시에만 커밋 후 자체 빌드를 트리거하고 trigger 소스·타입으로 필터링해 순환 트리거(loop) 차단

기술 스택

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석 전 주기 단독 설계·구축) 2024.11 ~ 현재 (누적 약 18개월)

로그가 분산된 온프레미스 환경에서 시작해 중앙 집중식 로깅을 3단계로 고도화(ELK → EFK → ECK Operator)하고, 수집기 데이터 무결성·HA를 강화한 뒤, 축적된 EFK 로그 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 재활용했습니다. build → operate → improve → analytics 로 이어지는 관측 플랫폼 전 주기를 단독으로 설계·운영했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

온프레미스 환경에 분산된 서버·컨테이너 로그로 장애 원인 파악이 느리고 통합 모니터링 체계가 없어, 중앙 집중식 로깅을 3단계로 고도화했습니다.

Phase 1 (ELK): Elasticsearch + Logstash + Kibana + Filebeat 구축 — Filebeat 수집 → Logstash JSON 파싱·필드 매핑 → Elasticsearch 저장 → Kibana 대시보드 실시간 모니터링·검색.

Phase 2 (EFK 전환): Logstash JVM 메모리 부담(1GB+)·Filebeat Helm 생태계 불일치·커스텀 확장성 한계를 해소하기 위해 Fluent Bit + Fluentd로 전환. C 기반 경량 Fluent Bit DaemonSet이 K8s 메타데이터 자동 태깅과 함께 수집, Fluentd가 멀티 output 가공/필터링 후 Elasticsearch 전달. 차트 업그레이드 자동화 스크립트로 upstream 차트 업그레이드 시 커스텀 PVC 템플릿 자동 보존.

Phase 3 (ECK Operator + Stack 9.0): Elastic 공식 Helm Chart 업데이트 2년 이상 중단과 CRD 기반 선언형 관리 주류화에 따라 Stack 8.5.1 → 9.0.0 major upgrade + ECK Operator 3.3.2를 elastic-system ns에 도입. ES/Kibana를 Custom Resource(CR wrapper 로컬 차트)로 재정의, monitoring → logging ns 병렬 배포 후 cutover로 무중단 전환, TLS 인증서 자동 회전(1년/30일)·elastic 패스워드 Helm template 관리·CR spec.version 한 줄 변경 롤링 업그레이드 체계 확립.

Phase 4 (AWS(EKS) 로깅 확장): 온프레미스 EFK 스택을 AWS(EKS)용 `-aws` 스택 5종(elasticsearch-aws / kibana-aws / fluent-bit-aws / fluentd-aws / eck-operator-aws)으로 template화해 동일 로깅 체계를 AWS에 이식, ILM hot/warm/cold 60일 삭제 + S3 스냅샷 SLM 90일 보관을 IRSA 기반으로 자동화(정적 키 없음).

트러블슈팅: Logstash null 바이트(\x00) 파싱 깨짐(mutate gsub 전처리), 필드 매핑 충돌 mapper_parsing_exception(인덱스 템플릿 명시 매핑+타입 변환 필터), 디스크 워터마크 초과 read-only(ILM 7일 자동 삭제), [EFK] Fluent Bit NFS WAL 쓰기 실패(커스텀 PVC 패치 자동 보존)·Fluentd buffer overflow 로그 유실(chunk_limit_size·flush_interval 튜닝+overflow_action=block), [ECK] Kibana 9 secure-cookie HTTP 로그인 차단 (tls.disabled+publicBaseUrl http://+secureCookies=false), Fluentd ES9 variant 부재(fluent-plugin-elasticsearch 5.4.4 suppress_type_name:true로 ES8 이미지에서 ES 9.0 호환+active marker E2E 검증), 3.3.x stackconfigpolicy-controller가 managedNamespaces 밖 리소스 17분마다 GET reconcile error(managedNamespaces:[] cluster-wide watch 우회).

성과: 로그 검색 시간 90% 단축·MTTR 개선, EFK 전환으로 Logstash JVM(1GB+) 부담 해소, 일 평균 1GB 로그 실시간 처리·90일 보관, ECK 전환으로 TLS·패스워드·StatefulSet 업그레이드 수동 부담 제거 + Stack 9.0 major upgrade로 2년 누적 격차 해소, Operator secureMode + ServiceMonitor로 controller-runtime 메트릭 kube-prometheus-stack 자동 등록, AWS(EKS)에 `-aws` 스택 5종 template화로 온프레미스 동일 로깅 이식(ILM 60일 삭제·S3 SLM 90일, IRSA 기반).

기술 스택

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

3-2. Fluent Bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 2026.05)

Phase 3 ECK 전환 후 Fluent Bit Pod 재시작 시 tail input in-memory offset 소실로 동일 로그 재색인/누락 위험이 있었고, 단일 replica + PDB+preStop hook 부재로 노드 drain 시 수집 공백·수집기 자체 알림 규칙 부재로 감지되지 않는 장애 가능성이 있었습니다.

Tier 1 (데이터 무결성): tail input에 SQLite offset DB(`db /var/log/flb-storage/tail.db`) + 전용 PVC(2Gi) 도입으로 Pod 재시작 후 offset 보존, helmfile revision 4에서 재색인 0·중복 0 검증.

Tier 2 (가용성): PodDisruptionBudget(`minAvailable=1`) + preStop hook(`sleep + flush`) + liveness/readiness probe(/api/v1/health)로 노드 drain·롤링 업데이트 중 수집 공백 최소화.

Tier 3 (모니터링): PrometheusRule 7개 alert(input/output 실패율, retry 누적, fluentd queue 압력, fluent-bit Pod down, SQLite DB write fail 등) + Slack alert routing으로 감지되지 않는 장애 사전 차단·운영자 hand-off.

Tier 4: HA 확장 옵션 3종(입력 path 분할/사이드카/Vector 멀티 수집기) 비교 문서화.

성과: Pod 재시작 시 재색인 0·중복 0으로 로그 무결성 보장, PDB+preStop으로 수집 공백 최소화 MTTR 직접 기여, 오피스버빌리티 stack 자기 모니터링 체계 확립, HA 옵션 3종 문서로 향후 의사결정 비용 절감.

기술 스택

Fluent Bit (tail / SQLite offset), Kubernetes PVC / StorageClass, PodDisruptionBudget, Prometheus Operator (PrometheusRule), Helmfile, Fluentd (buffer tuning)

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

EFK 로깅 스택이 장애 대응·로그 검색 용도로만 쓰이고 게임의 신규 유저·리텐션·이탈을 정량 관측할 제품 분석 지표가 없어, 별도 분석 SaaS 없이 기존 로그 파이프라인을 재활용했습니다. EFK 위에 Elasticsearch Transform(5분 연속 실행)으로 원시 이벤트를 per-user Cohort 인덱스(first_seen / active_dates / max_cleared_chapter)로 피벗하고 KST runtime field로 일자 경계를 보정했습니다. 이 인덱스 위에 12패널 Kibana 대시보드(NU/DAU/WAU/MAU KPI, NU·DAU 추세, D+1~D+30 리텐션 곡선, 일별 Cohort 리텐션 테이블, 챗터 분포)를 구성하고, 대시보드·Transform 정의를 환경별로 템플릿화하는 가이드를 문서화했습니다.

성과: 장애 대응 중심 로그 스택을 제품 분석 플랫폼으로 확장 — 별도 SaaS 없이 신규 유저·리텐션·챗터 진행 정량 관측, D+1~D+30 리텐션 곡선·일별 Cohort 테이블 제공, 환경별 템플릿으로 재사용 자산화.

기술 스택

Elasticsearch Transform, Kibana, EFK Stack, Runtime Field (KST), ECK Operator, Kubernetes

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% 2025.02 ~ 현재

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins 빌드 완료 시 Slack에 QR 코드와 함께 자동 알림. QA팀 전달 시간 90% 단축

기술 스택

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab Webhook + Google Drive API로 Push 시 자동 동기화. 문서 관리 시간 80% 감소. 업로드 잡은 rclone `--checksum` 으로 미변경 파일 스킵 + sparse-clone으로 대상 디렉토리만 클론하도록 점진적으로 최적화

기술 스택

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. 내부 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama + Open WebUI를 Kubernetes에 배포. 개발팀 15명 · 일 평균 100건 쿼리 처리, 사용 가이드 문서 작성 + 개발팀 온보딩 안내로 도구 정착 주도.

외부 AI SaaS 구독 대체로 월 약 \$100 절감. 사내 보안 가이드라인 100% 준수 및 외부 데이터 유출 리스크 제거

기술 스택

Ollama, Open WebUI, Kubernetes, NFS, GPU Server

4-4. Git 저장소 미러링 도구 개발 + Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 2026.06)

Go 기반 양방향 Git 미러링 도구(git-bridge) 개발. 유닛 테스트 + GitHub Actions CI/CD 파이프라인으로 코드 품질 확보, CodeCommit/GitLab/GitHub 간 자동 동기화로 저장소 10개·일 평균 30건 동기화 100% 자동화.

Phase 2(2026.05)로 `POST /retry/mirror` 엔드포인트 + Bearer token 인증(`RETRY_API_TOKEN` , webhook secret과 분리, constant-time compare)을 추가하여 webhook secret 노출 없이 안전한 수동 재시도 경로 확보. direction `auto / source-to-target / target-to-source` 3종 + repo별 `retry_direction` override + 리포지토리별 Slack webhook 환경 변수로 알림 채널 분리 지원. 11개 신규 테스트를 추가해 CI 테스트 게이트 통과

Phase 3(2026.06)에서 조율되지 않은 다수의 force-push가 공유 브랜치를 옛 스냅샷으로 되돌리던 비-fast-forward 회귀의 근본 원인을 분석하고, ref 패턴별로 동기화 방향을 고정하는 `ref_overrides` 를 설계. repo는 양방향으로 유지하되 특정 브랜치만 단방향으로 고정해 역방향 echo로 브랜치가 되돌아가는 일을 원천 차단하고, push 범위를 트리거된 ref로 좁힘(`--all` 제거, override repo만 적용 → 기존 repo 동작 그대로 유지) + delete 이벤트도 같은 방향으로 제한 + fail-open 폴백 적용. 테스트 전용 repo에서 6/6 런타임 검증 PASS(프로덕션 repo 격리·영향 없음), main에 커밋 4개(feat / fix / docs / chore) 반영

기술 스택

Go, AWS SQS, GitLab Webhook, GitHub Webhook, HTTP Bearer Auth, Slack Webhook, Kubernetes, Docker

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

Go 기반 경량 정적 파일 서버(static-file-server)를 직접 개발하고 GitHub Pages에 Helm repository를 운영. 다크모드 디렉토리 리스팅 UI, 파일 프리뷰 (이미지/비디오/PDF/텍스트), 검색·URL 해시 공유, 다중 선택 + ZIP 일괄 다운로드, Gzip, Prometheus 메트릭, JSON 로깅, APK Content-Type 자동 설정 기능 제공.

자체 Helm chart로 Kubernetes에 배포(NFS 스토리지)하고 기존 OSS 이미지(halverneus/static-file-server)를 완전 대체. 일 평균 30건 다운로드 · 주 5회 (매일 빌드) APK 배포 트래픽 처리. Apache-2.0 오픈소스 공개.

기술 스택

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (단독 설계 — 모니터링·업그레이드 자동화·Helm 관리 프레임워크·NGF 마이그레이션·OSS Helm 차트 3종·스토리지 성능 최적화·Shell→Python 마이그레이션, 자동화 4 스크립트로 표준 절차 자산화 → 팀 재사용 가능) 2025.12 ~ 현재

클러스터 가시성, 운영 안정성, 네트워크 고도화, 스토리지 성능, 자동화 코드 품질을 목표로 모니터링 체계를 고도화하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다.

NGF 마이그레이션과 ECK Operator 도입의 산출물을 OSS Helm 차트로 공개하고, control-plane / DB 노드의 SSD 마이그레이션 및 빌드 I/O 격리로 스토리지 성능 병목을 해소했으며, 인프라 배포/업그레이드 파이프라인을 단계별(wave) 순차 자동화 + Shell → Python 마이그레이션으로 정비했습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반으로 다수의 커스텀 알림 규칙과 Grafana 대시보드를 설계하고, MySQL·Redis·Elasticsearch·PostgreSQL exporter를 통합했습니다. 물리서버와 VM에 Ansible로 node-exporter를 자동 배포하고, Cilium CNI agent/operator 메트릭을 kubernetes_sd_configs로 수집. Alertmanager에서 severity 기반 Slack 라우팅과 inhibit rules를 구성해 알림 품질을 확보했습니다.

(사내 메트릭 보관·exporter 자유 확장·SaaS 구독료 부담 없음이 우선이라 Datadog / New Relic 같은 SaaS Observability 대신 kube-prometheus-stack 을 채택.)

기술 스택

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드 자동화 스크립트로 사전 검증·백업·호환성 확인, 다단계 헬스체크 스크립트로 검증 시간 90% 단축.

Helm 차트 업그레이드 프레임워크 + 4가지 캐노니컬 템플릿 동기화 도구(check로 CI drift 감지, apply로 일괄 전파)로 스크립트 본문 일관성 자동 유지.

K8s 인증서 자동 갱신 스크립트 + crontab(노드 전만, 6개월마다 새벽 3시) 주기 실행으로 인증서 만료로 인한 API Server 장애 사전 방지 (kOps는 AWS-only, Cluster API 는 온프레미스 bootstrap 이 무거워, Ansible에 익숙한 팀 운영에 맞춰 Kubespray 유지 + 자동화 레이아웃 추가.)

기술 스택

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

Ingress-nginx 컨트롤러 다수 인스턴스(기본 + public-a~j, 클래스별 MetalLB LoadBalancer IP 고정)를 공식 후속 컨트롤러 NGF 2.x 기반 단일 컨트롤플레인 + 클래스별 Gateway CR 구조로 전환. MetalLB IP를 그대로 유지하여 DNS/방화벽 변경 없이 병렬 배포 + 점진 cutover 방식 채택(클래스당 실 다운타임 30~60초).

Phase 0~7+ 전 단계(MetalLB 풀 확장 → NGF 설치 → 관측 스택 → HTTP-only/ApplicationSet → HTTPS 강제 앱(Harbor·Vaultwarden) → IP swap cutover → Ingress 정리)로 진행. 어노테이션을 Gateway API + NGF CRD(HTTPRoute filters, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy)로 1:1 매핑, HTTPRoute 차트 values 스키마를 여러 차트에 통일해 ApplicationSet 일괄 전환에 재사용. self-signed wildcard 인증서 1장(내부 도메인 전체, 10년)으로 애플 TLS Secret을 통합해 수동 갱신 부담 50% 감소.

기술 스택

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. OSS Helm 차트 3종 공개 — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

nginx-gateway-cr: NGF 마이그레이션(5-3) 산출물에서 추출한 local CR chart를 Gateway / NginxProxy / ReferenceGrant / ServiceMonitor / PodMonitor 5종 테넌트 리소스를 배포하는 범용 차트로 공개했습니다. multi-Gateway 친화적 스키마(`gateways[]` 배열 + per-gateway override), NGF 2.x 권장 PodMonitor 템플릿(controller + dataplane)을 포함했습니다.

elasticsearch-eck / kibana-eck: elasticsearch-eck(11 ES 템플릿) + kibana-eck(10 Kibana 템플릿)으로 CR·Secret·HTTPRoute·Ingress·BackendTLSPolicy·ServiceMonitor·NetworkPolicy를 단일 values로 배포했습니다. `values.schema.json` (draft-07) 입력 검증 + Minimal/HA 프리셋 + 환경별 storageClass 매핑을 README에 명시했습니다.

배포 통일: 세 차트 모두 OCI(ghcr.io/somaz94/charts) + gh-pages Helm 레포 + ArtifactHub(networking / monitoring-logging, Apache-2.0) 세 경로로 동시 배포했습니다. OCI 차트 버전 추적용 canonical 템플릿도 함께 작성했습니다.

무중단 전환: mgmt 클러스터 로컬 차트 → OCI 차트 전환 시 CR·PVC·Secret 이름 보존으로 cluster_uuid 불변, ES pod 재시작 0회·Kibana rolling 1회로 데이터 손실 없이 전환을 완료했습니다. HA 롤링 kind 검증 3/3 PASS, 실 클러스터 설치 Elasticsearch 71초·Kibana 57초 Ready.

기술 스택

Helm Chart, Gateway API, NGINX Gateway Fabric, ECK Operator, Elasticsearch, Kibana, Prometheus Operator, OCI Registry, ArtifactHub, Kubernetes

5-5. 스토리지 성능 최적화 — etcd / DB SSD 마이그레이션 + 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

control-plane VM과 게임 DB 워커(game-DB worker VM)가 HDD에서 동작하며 GitLab Runner buildx 작업과 I/O 경험 발생 → etcd WAL fsync p99 약 2초·apiserver pods GET p99 약 7초·게임 DB COMMIT latency 9~14초로 폭증해 실 플레이가 불가능. game-DB worker VM 디스크 포화로 root cause 특정.

Step 1: MySQL `innodb_flush_log_at_trx_commit=2` 등 6개 파라미터 튜닝으로 즉시 완화.

Step 2 (control-plane VM SSD migration): KVM qcow2 sparse copy(11GB, 6분 50초) + virsh XML edit으로 다운타임 8.5분(예상 15분 대비 단축) 완료.

Step 3 (game-DB worker VM DB/Redis SSD migration): game-db (MySQL) + valkey-redis 3환경 총 35GB stop → backup → SSD migration → restore → local-path PV 재바인딩으로 데이터 손실 없이 이전.

Step 4 (build-only VM): 별도 물리 서버에 4 vCPU/16GB/150GB SSD VM을 cloud-init으로 프로비저닝, kubespray로 join, `node-role=build` label + `dedicated=build:NoSchedule` taint 부여로 buildx 격리. 4종 자동화 스크립트로 자산화.

포스트모템 · 재발 방지: MAC 60진수 파싱 함정 → VM 검증 스크립트의 MAC 일치 게이트화, etcd leader-election 5xx → 마이그레이션 runbook 'etcd quorum 회복 확인' 게이트 추가, 4종 자동화 스크립트 + 설정 파일을 표준 절차 자산화로 차후 워커 노드 추가 시 재사용.

성과: etcd WAL fsync 1760ms → 3.88ms (454배), etcd backend commit 1810ms → 3.77ms (480배), apiserver pods GET p99 6520ms → 23ms (283배), 게임 DB COMMIT 9~14초 → ms 수준 회복, 빌드 격리 VM 검증 11 PASS / 0 FAIL

기술 스택

KVM / libvirt, qcow2, virsh, etcd, Prometheus / PromQL, MySQL, Redis (valkey), kubespray, cloud-init, GitLab Runner

5-6. 인프라 배포/업그레이드 자동화 + Shell → Python 마이그레이션 (2026.03 ~ 현재)

사내 Kubernetes 인프라 monorepo 25개 컴포넌트가 helmfile + shell로 수동 관리되며 multi-env 분기·복잡 YAML 패칭에서 한계. **배포 자동화**로 CI 파이프라인을 6 wave(bootstrap → core → security → db-redis → observability → apps)로 구성해 MR diff 감지 → 변경 컴포넌트 helmfile diff/apply, manual gate로 단계별(wave) 순차 배포 표준화. CI script 10종(helmfile-changed-dirs, diff-component, apply-component, auto-upgrade, render-mr-body 등) 신규.

업그레이드 자동화로 CI가 25개 컴포넌트 신버전을 감지해 자동 MR 생성, 본문에 변경 chart-values diff·breaking change 노트 자동 렌더링.

Shell → Python 마이그레이션으로 8종 canonical template + 동기화·백업 스크립트 + CI script 6종을 stdlib only로 재작성. 25개 컴포넌트별 업그레이드 모듈 + 758 unittest 정비. helmfile-tools image(프로젝트 7) 적용으로 **CI 환경 준비 시간 5분 → 약 10초(약 95% 단축)**, 로컬(macOS venv) + CI 컨테이너 동일 코드 보장

이 helmfile push 배포 모델은 이후 5-7의 ArgoCD pull-GitOps 컨트롤플레인으로 전환되어, push 기반 배포를 pull 기반 선언 관리로 대체 (배포 모델 진화)

기술 스택

Python 3.10+ (stdlib only), unittest, GitLab CI/CD, Helm / Helmfile, Git automation, yq

5-7. helmfile Push → ArgoCD Pull-GitOps 전환 (App-of-apps 컨트롤플레인, 2026.06 ~ 현재)

플랫폼 릴리스를 helmfile push로 배포하다 보니 배포 주체가 CI 러너에 묶여 있었고, 멀티클러스터 확장 시 클러스터별 드리프트를 선언적으로 수렴시킬 pull 기반 GitOps 컨트롤플레인이 필요했습니다. Matrix + Git files 제너레이터 ApplicationSet이 각 컴포넌트의 `argocd/*.yaml` 마커를 읽어 Application을 자동 생성하도록 구성해 온프레미스 16 + AWS 8 = 24개 플랫폼 릴리스를 2개 클러스터에 등록했습니다.

무중단 adoption: 기존 helm 릴리스를 재생성 없이 adopt(`resourceTrackingMethod=annotation`)하고 helmfile deploy-wave를 ArgoCD sync-wave로 1:1 매핑해 배포 순서 보존.

App-of-apps 부트스트랩: root → {app,infra}-projects(wave -1) → {app,infra}-appsets(wave 0) 계층으로 부트스트랩하고, prune-cascade 안전장치로 컨트롤플레인은 `prune:false + selfHeal:true` ·leaf만 `prune:true` 로 분리해 대량 삭제 차단, 업그레이드는 argocd-pin 패턴으로 관리.

안정화·최소권한: ArgoCD resource.exclusions로 컨트롤플레인이 대량 생성하는 리소스(Endpoints/EndpointSlice/coordination.k8s.io Lease/Authz/Authn)를 추적 제외해 watched events·UI clutter·컨트롤러 부하를 완화(온프레미스·AWS 양쪽 적용)하고, 테넌트 AppProject의 `clusterResourceWhitelist` / `namespaceResourceWhitelist` 를 * 대신 차트 렌더 kind만 명시(클러스터 스크오프는 Namespace·PersistentVolume만, ClusterRole/CRD 미부여, infra만 `*/*`)해 최소 권한·blast-radius를 축소했습니다.

성과: 2개 클러스터 24개 릴리스(온프레미스 16 + AWS 8)를 단일 컨트롤플레인에서 선언 관리, 재생성 없는 adoption으로 무중단 전환, prune-cascade 안전장치로 GitOps 대량 삭제 리스크 차단, resource.exclusions·AppProject 최소권한 whitelist로 UI clutter·컨트롤러 부하 완화 및 blast-radius 축소 (신규 kind는 disallowed-resource sync 실패로 조기 검출).

기술 스택

ArgoCD, ApplicationSet (Matrix / Git files generator), GitOps, Helm / Helmfile, Multi-cluster, Kubernetes

프로젝트 6: 인프라 보안 체계 구축

기여도 100% 2026.04 ~ 현재

6-1. Vaultwarden 비밀번호 관리 시스템 (1주, 2026.04 ~ 현재)

Vaultwarden을 K8s에 Helm 배포, GitLab SSO 연동, 조직/컬렉션 기반 접근 제어. 자동 백업(날짜별, 30일 retention) + 대화형 restore. 사용자 70명 · 시크릿 50건 중앙 관리 체계로 전환

기술 스택

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

6-2. Keycloak Operator 기반 중앙 인증(SSO) 시스템 도입 (2026.04 ~ 현재)

ArgoCD / Harbor / Vaultwarden이 각각 GitLab을 OIDC IdP로 직접 사용하던 구조를 중앙 broker(Keycloak)로 일원화. Keycloak Operator + KeycloakCR + KeycloakRealmImport를 PostgreSQL 백엔드와 함께 도입하고, GitLab을 Identity Provider로 brokering(기존 계정 그대로 로그인). Realm / Client / IdP / Group / Mapper는 `kcadm.sh` bootstrap 스크립트로 codify. ArgoCD / Harbor / Vaultwarden이 GitLab에 직접 연동하던 OIDC를 Keycloak을 거쳐도록 전환. GitLab group(`server` , `global-admin`)을 Keycloak group mapper로 brokering한 뒤 ArgoCD role mapping에 연결해 권한 운영의 단일 기준을 Keycloak으로 통일.

성과: ArgoCD/Harbor/Vaultwarden OIDC 통합 완료(다운타임 0, 검증 PASS), 도입 과정에서 발견된 5가지 함정(IdP providerId/scope/mapper config/Operator idempotency/Harbor user mapping)을 plan 문서에 정리해 재발 방지, 향후 LDAP federation/MFA/세션 정책 중앙화의 기반 마련

기술 스택

Keycloak Operator, KeycloakCR / KeycloakRealmImport, PostgreSQL, OIDC / OAuth2, kcadm.sh, ArgoCD, Harbor, Kubernetes Gateway API

6-3. GitOps 시크릿 관리 — External Secrets Operator · Sealed Secrets (2026.05 ~ 현재)

External Secrets Operator로 ClusterSecretStore ↔ AWS Secrets Manager를 연결해 DB 비밀번호 평문 values를 ExternalSecret 참조로 전환 (ExternalSecret을 Deployment보다 1 sync-wave 먼저 동기화). Harbor image-pull secret은 cluster-wide SealedSecret으로 암호화해 admin-only AppProject에서 관리.

ArgoCD 알림은 notify-channel 라벨 라우팅으로 Slack 채널 분리.

기술 스택

External Secrets Operator, AWS Secrets Manager, Sealed Secrets, ArgoCD, Helm, Kubernetes

프로젝트 7: 공용 Toolchain Image + 자동 버전 사이클

기여도 100% (아키텍처 설계 · CI 표준화 · 자동화 파이프라인 개발) 2025.09 ~ 2026.05

문제

GitLab CI 표준화 범위 안의 15개 repo(1개 template provider + 14개 consumer)에서 각 잡이 alpine 베이스에 helm·helmfile·kubectl·crane·aws-cli 등을 매번 설치하면서 첫 번째 잡 준비 시간이 평균 5분에 달했고, 도구 버전이 14개 consumer repo에 흩어져 있어 신버전 추가 시 일괄 갱신이 사실상 불가능했습니다. `:latest` 태그 의존으로 재현성도 떨어졌습니다.

접근 전략

Layer 1 (Mirror): 외부 레지스트리의 도구 이미지를 `crane copy` 로 사내 Harbor에 multi-arch manifest 보존 미러링.

Layer 2 (Custom): Layer 1 위에 도구가 사전 설치된 fat image 7종(helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools) 빌드.

Tier 1~2 SSOT: `.toolchain_build_custom` 공용 template + `T00LCHAIN_*_TAG` 중앙 변수 도입으로 consumer repo는 image tag를 직접 추적하지 않도록 간접화.

공용 CI 템플릿 SSOT: repo마다 복붙하던 CI 잡을 중앙 공용 CI 템플릿 repo로 추출해 각 repo가 `include` 로 참조하도록 전환. auth(키리스 AWS 인증) / build(kaniko) / deploy(ArgoCD) / backup(Google Drive) 잡 템플릿과 공용 앵커(git-ssh-rules·job-config·packages)를 단일 소스로 관리해 복붙 드리프트 제거.

Phase 4 자동 사이클: cron 트리거로 다음 4단계가 자동 진행되어 사람 click 2회로 연쇄 전파 완성:

- (1) upstream 신버전 감지
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 동기화 auto-MR
- (4) 14개 consumer 자동 전파

성과

- CI 첫 번째 잡 준비 시간 5분 → 약 10초 (약 95% 단축), 14개 consumer repo 도구 버전 일관성 100% 확보 (template provider 포함 총 15개 repo)
- 버전 거버넌스 4-tier 구조(ARG → 중앙 변수 → consumer 간접화 → lint drift 감지)로 흩어진 12+ 버전 포인트 압축
- Phase 4 cron 자동 사이클로 5+ 도구의 신버전 연쇄 전파를 사람 click 2회로 완료, 매주 신버전 추적 운영 부담 사실상 제거. minor-guard 정책으로 호환성 깨짐 사전 차단
- 25개 컴포넌트를 리스크 티어(T1 stateless 주간 / T2 platform 격주 / T3 stateful 월간 + pre-flight)로 분류한 자동 업그레이드 파이프라인 — 신버전 감지 → 컴포넌트별 자동 MR → Slack 알림 → wave 순차 적용, MAJOR_PIN 가드와 T3 atomic 롤백으로 안전성 확보
- docker:dind 사이드카 + docker buildx(docker-container driver)로 amd64 + arm64 멀티아치 매니페스트 리스트를 빌드하고, 외부 이미지 (alpine·node·golang·rclone·awscli·docker)를 crane으로 Harbor에 미러링해 외부 레지스트리 의존 제거

기술 스택

GitLab CI/CD, Docker Buildx (multi-arch), Docker BuildKit (docker-container driver), crane, Kaniko, Harbor, Helm / Helmfile, Bash / yq, Slack Webhook

프로젝트 8: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (그린필드 EKS 단독 설계·구축) 2026.06 ~ 현재 (구축·운영 중)

문제

라이브 게임 estate(AWS QA·Prod)와 별개로, 신규 외부 퍼블리싱 게임은 온프레미스 개발 클러스터 단일 체계로만 운영 중이었고, 프로덕션 런칭을 위해 확장성·가용성을 갖춘 별도 AWS EKS 프로덕션 환경이 필요했습니다. 온프레미스만으로는 글로벌 트래픽 대응과 오토스케일링에 한계가 있었습니다.

접근 전략

eu-central-1 리전에 그린필드 EKS 클러스터를 구축해 온프레미스 개발 + AWS 프로덕션의 멀티클러스터 하이브리드로 확장하고, 플랫폼 컴포넌트 약 11종을 전부 GitOps(ArgoCD)로 선언 관리했습니다.

오토스케일링·네트워킹: Cluster Autoscaler 대신 Karpenter를 채택(여러 스팟 family 통합·빠른 provisioning)해 9개 스팟 인스턴스 family에 걸친 스팟 오토스케일링과 게임 서버 전용 on-demand NodePool을 구성하고, AWS Load Balancer Controller(ALB/NLB) + ExternalDNS(Route53) + wildcard ACM으로 인그레스·DNS·TLS를 자동화했습니다.

로깅·시크릿: eck-operator 기반 EFK 스택(Elasticsearch 3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)을 HA로 구성하고, External Secrets Operator로 시크릿을 외부화했습니다. 모든 ServiceAccount 인증은 IRSA / Pod Identity로 처리해 정적 키를 제거했습니다.

인증·배포: ArgoCD에 GitLab OAuth SSO + 게임 스크립트 RBAC를 연동하고, 클라이언트 아티팩트는 Jenkins → S3 + CloudFront로, 서버 매니페스트는 S3 transit 버킷 + SSM SendCommand로 bastion의 EFS 마운트에 전달했습니다. bastion을 Name 태그로 동적 탐색해 SSH 키 없이 배포했습니다.

트러블슈팅

- 게임 프로덕션 Pod가 실제 사용량 대비 과도한 메모리(2Gi)를 예약해 노드가 불필요하게 증설되는 node sprawl 발생 → 실측 기반으로 memory requests를 512Mi로 조정해 노드 증설을 정상화하고 스팟 비용 효율 확보
- 롤링 배포 중 ALB가 아직 준비되지 않은 Pod로 트래픽을 전달하는 문제 → ALB Pod readiness gate + PodDisruptionBudget + graceful drain을 조합해 무중단 배포 확보

성과

- 온프레미스 개발 단일 체계 → 온프레미스 + AWS 프로덕션 멀티클러스터 하이브리드로 확장, 신규 외부 퍼블리싱 게임의 클라우드 프로덕션 런칭 기반 확보
- 약 11종 플랫폼 컴포넌트를 100% GitOps(ArgoCD)로 선언 관리, IRSA / Pod Identity로 정적 키 없는 인증 체계 확립
- SSM SendCommand 기반 키리스 배포로 bastion SSH 키 관리 부담과 키 유출 리스크 제거
- ArgoCD 앱 마커와 프로덕션 CI/CD 전환 문서로, 단독 구축이지만 팀이 동일 절차로 운영·재구축 가능하도록 인수인계 경로 확보

기술 스택

AWS EKS (eu-central-1), Karpenter, AWS Load Balancer Controller, ExternalDNS, External Secrets Operator, ECK Operator / EFK, ArgoCD, IRSA / Pod Identity, AWS SSM (SendCommand), S3 / CloudFront, Kubernetes

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임 및 블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며, AWS에서 GCP로 대규모 클라우드 마이그레이션 프로젝트를 총괄했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 설계·마이그레이션 총괄 70% (나머지 30% 는 서버 개발팀 application 마이그레이션 협업), CI/CD 파이프라인 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용 지속 상승(월 \$10,000+, NAT Gateway·데이터 전송 비용 高), GCP Startup Program 참여 기회, 멀티 리전 게임 서비스를 위한 GCP 글로벌 LB 확보

접근 전략

Shared VPC (Host / Service Project 분리) 와 Workload Identity Federation 을 도입해 GitHub Actions 가 서비스 계정 키 없이 임시 토큰으로 GCP 리소스에 인증하도록 구성. DNS 는 서브도메인 위임 (Hosting.kr → AWS Route53 → GCP Cloud DNS) 으로 사전 검증 후, 최종 NS 변경 시점에만 점검 공고 후 cutover.

Filestore 의 SSD 최소 2.5TB 비용 문제는 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 우회. Cloud Armor 로 region 차단 + IP 화이트리스트 WAF 정책을 구성하고, Cloud CDN 의 URL Map 을 HTTP / HTTPS 두 개로 분리하여 HTTP → HTTPS 강제 리다이렉트와 정적 자산 캐싱 동시 활성화.

트러블슈팅

- AWS ALB → GCP 글로벌 LB 전환 시 AWS ALB는 헬스체크 실패 백엔드에도 트래픽을 통과시키지만 GCP LB는 차단하는 차이로 라우팅 이슈. GCP NEG 구조 분석 후 헬스체크·백엔드 재구성
- GKE Ingress + BackendConfig 의 healthcheck requestPath 응답 누락으로 503 발생. 해당 경로 응답 보장 + ManagedCertificate / FrontendConfig 정렬로 해결
- CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드 실패. URL Map 을 HTTP / HTTPS 두 개로 분리 (`default_url_redirect.https_redirect=true`) 후 해결
- DNS를 Cloud DNS로 위임 후 Google Search Console 도메인 미등록 문제 발생. Search Console 발급 verification TXT 레코드를 Cloud DNS에 추가해 소유권 검증 후 재등록으로 해결

성과

- 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 배포 시간 50% 단축, 롤백 95% 감소

- 전체 GCP 인프라 Terraform IaC화 (TFLint/Checkov 정적 분석, Terratest 단위 테스트 도입), Git 커밋 기반 배포 이력 100% 추적
- 리소스 복사 방식으로 다운타임 최소화, 최종 DNS 전환은 게임 서비스 특성을 반영해 약 2시간 내외 점검 공지 후 전환
- Workload Identity Federation 도입으로 GitHub Actions 의 GCP 서비스 계정 키 관리 부담 제거 및 키 유출 리스크 원천 차단

기술 스택

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하여 분석가가 매일 2~3시간 소요, 휴먼 에러로 인한 데이터 누락 빈번 발생

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 데이터 소스를 BigQuery 로 적재 (MongoDB 는 Dataflow ETL, CloudSQL 은 직접 connection, GA/Dune 은 API) 후, Cloud Functions + Cloud Scheduler 가 Daily / Monthly 단위로 자동 트리거하고 Google Sheets API 로 분석가용 시트에 자동 적재. BigQuery 단의 중복 데이터는 별도 Cloud Function 으로 정기 제거.
전체 인프라 (Dataset · Cloud Function · Scheduler · Storage · Artifact Registry) 는 Terraform IaC 로 관리.

성과

- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0%

기술 스택

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Docker

프로젝트 3: PLG Stack 기반 모니터링 시스템 구축

기여도 100% 4개월 (2024.04 ~ 2024.07)

문제

Kubernetes 클러스터·애플리케이션 실시간 모니터링 체계가 없어 사후 대응만 가능했고, 장애 원인 파악에 과도한 시간이 소요됨

성과

- SLI/SLO 수립 및 Alertmanager 알람 노이즈 최적화, 장애 사전 감지 체계 확립, 장애 대응 시간 단축, 서비스 가용성 향상

기술 스택

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했고, 금융권 및 공공기관 대상 온프레미스 클라우드 인프라 설계·구축 및 운영 교육을 직접 진행했습니다.

주요 프로젝트: PaaS 솔루션 구축 · DP사 교육 · 인증서 자동화 · Ansible 구성 관리

성과

- 5개 고객사에 PaaS 솔루션 구축, Kubernetes HA 99.9% 가용성
- DP사 운영팀 Kubernetes 자체 운영 체계 확립 (6개월, 수강자 평균 10명 · 회당 4시간)
- 인증서 갱신 주기 10배 연장 (1년 → 10년), 연간 50시간 운영 부담 절감
- Ansible 기반 서버 프로비저닝 및 구성 관리 자동화로 환경 불일치(Configuration Drift) 이슈 제로화

기술 스택

Kubernetes, OpenStack, Ceph, Ansible, kubeadm, Rocky Linux/CentOS

이테크시스템

2022.01 ~ 2022.04

System Engineer

Hardware Server 및 SAN 스토리지 구축, DevOps 분야로 커리어 전환을 위해 이직.

오픈소스 프로젝트

Kubernetes CRD

[k8s-namespacesync](#) — Kubernetes 네임스페이스 리소스를 클러스터 간 자동 동기화
[helios-lb](#) — Kubernetes용 커스텀 로드밸런서 컨트롤러
[network-policy-generator](#) — Kubernetes NetworkPolicy 자동 생성 컨트롤러

GitHub Actions

[Compress Decompress Action](#) — 워크플로우 내 파일 압축/해제
[Image Tag Updater](#) — YAML/Helm 파일 이미지 태그 자동 업데이트
[Go Git Commit Action](#) — Go 기반 Git 커밋 생성 및 푸시
[Extract Commit Action](#) — Git 커밋 SHA, 메시지, 작성자 정보 추출
[Multi Git Mirror Action](#) — 여러 Git 저장소 간 미러링 자동화
[Kube Diff Action](#) — Kubernetes 리소스 변경사항 비교 및 리포트
[Major Tag Action](#) — 메이저 버전 태그(v1)를 최신 semver 릴리즈로 자동 갱신
[Go Changelog Generator](#) — Conventional Commits 기반 체인지로그 자동 생성
[Environment Output Setter](#) — 여러 환경 변수·워크플로우 출력을 변환과 함께 설정
[Ternary Operator Action](#) — 최대 10개 조건 평가로 동적 출력 설정
[Contributors Action](#) — 저장소 데이터로 기여자 목록(HTML·MD·이미지) 자동 생성
[Kube Events Action](#) — Kubernetes 클러스터 이벤트 점검·필터링·PR 코멘트
[Helm Kustomize Lint Action](#) — Helm 차트 lint·검증(yamllint·helm lint·template·kubeconform)
[Helm OCI Push](#) — Helm 차트를 OCI 레지스트리(GHCR·ECR·Harbor·Quay)로 푸시

Ansible Galaxy

[Ansible K8s IAC Tool](#) — K8s 및 IaC 도구 자동 설치 컬렉션
[Ansible User Management](#) — Linux 사용자 및 SSH 키 관리 역할

DevOps Tools

[bash-pilot](#) — AI 기반 Bash 명령어 어시스턴트 CLI 도구
[git-bridge](#) — Git 저장소 간 미러링 및 동기화 CLI 도구
[static-file-server](#) — 디렉토리 UI·파일 프리뷰·검색·ZIP 일괄 다운로드 지원 Go 기반 정적 파일 서버
[kube-diff](#) — 매니페스트(YAML·Helm·Kustomize)를 라이브 클러스터와 비교하는 CLI
[kube-events](#) — Kubernetes 이벤트 조회·요약 CLI (그룹핑·필터·컬러)

Chrome Extensions

[Dev Toolkit](#) — 개발자 유틸리티 모음
[QRify](#) — URL/텍스트 QR 코드 즉시 생성

학력

충남대학교 행정학부 (학사편입)	2018.03 ~ 2020.08
국가평생교육진흥원 경영학사 (학점은행제)	2016.09 ~ 2018.02

자격증

정보처리기사	2021.11
AWS Certified Solution Architect - Associate (만료)	2021.11
네트워크 관리사 2급	2021.10
리눅스 마스터 2급	2021.10
컴퓨터활용능력 1급	2021.04
TOEIC Speaking Test - 130점/Intermediate Mid 3 (만료)	2021.02

스터디

기술 블로그 운영 (somaz.blog, 영어) — DevOps · Kubernetes · IaC 글로벌 독자 대상 정기 작성	2025.01 ~ 현재
기술 블로그 운영 (somaz.tistory.com, 한국어) — DevOps · Kubernetes · IaC 학습/운영 노트 꾸준한 작성	2023.04 ~ 현재
T101(Terraform) 스터디	2023.08 ~ 2023.10
AEWS(EKS) 스터디	2023.04 ~ 2023.06

PKOS(Kubernetes) 스터디	2023.01 ~ 2023.02
Cloud Security 교육	2022.02 ~ 2022.04
보안솔루션 운영 전문가 양성과정 (국비교육)	2021.07 ~ 2021.12